Front. Comput. Sci., 2020, 14(6): 146405
https://doi.org/10.1007/s11704-019-9120-2

RESEARCH ARTICLE

A post-processing method for true random number generators based on hyperchaos with applications in audio-based generators

Je Sen TEH (✉)[1], Weijian TENG[2], Azman SAMSUDIN[1], Jiageng CHEN[3]

1 School of Computer Sciences, Universiti Sains Malaysia, Penang 11800, Malaysia
2 School of Engineering and Technology, INTI International College Penang, Penang 11900, Malaysia
3 School of Computer, Central China Normal University, Wuhan 430079, China

© Higher Education Press and Springer-Verlag GmbH Germany, part of Springer Nature 2020

Abstract True random number generators (TRNG) are important counterparts to pseudorandom number generators (PRNG), especially for high security applications such as cryptography. They produce unpredictable, non-repeatable random sequences. However, most TRNGs require specialized hardware to extract entropy from physical phenomena and tend to be slower than PRNGs. These generators usually require post-processing algorithms to eliminate biases but in turn, reduces performance. In this paper, a new post-processing method based on hyperchaos is proposed for software-based TRNGs which not only eliminates statistical biases but also provides amplification in order to improve the performance of TRNGs. The proposed method utilizes the inherent characteristics of chaos such as hypersensitivity to input changes, diffusion, and confusion capabilities to achieve these goals. Quantized bits of a physical entropy source are used to perturb the parameters of a hyperchaotic map, which is then iterated to produce a set of random output bits. To depict the feasibility of the proposed post-processing algorithm, it is applied in designing TRNGs based on digital audio. The generators are analyzed to identify statistical defects in addition to forward and backward security. Results indicate that the proposed generators are able to produce secure true random sequences at a high throughput, which in turn reflects on the effectiveness of the proposed post-processing method.

Keywords audio, chaos theory, chaotic map, entropy, hyperchaos, post-processing, random number generator, security

Received April 5, 2019; accepted September 4, 2019

E-mail: jesen_teh@usm.my

1 Introduction

There are two types of random number generators characterized by predictability: a pseudorandom number generator (PRNG) generates the same sequence of random numbers for a given input seed whereas a true random number generator (TRNG) produces unpredictable random numbers. This unpredictability is obtained from physical sources such as radioactive decay, electrical noise [1], thermal fluctuations [2] and many more. Both generators play important roles in statistics, cryptography, scientific research, video games and lottery.

TRNGs are generally used for situations where unpredictability is of utmost importance such as generating cryptographic encryption keys or nonces whereas PRNGs are chosen for reproducibility. TRNGs are generally slower than PRNGs as they need to extract entropy from physical sources such as radioactive decay which usually require specialized hardware that are not widely available. The bulk of recent work on TRNGs are based on integrated circuits (ICs) such as application-specific integrated circuits (ASICs) or field-programmable gate arrays (FPGA) [3–12]. Although these TRNGs are highly efficient, these ICs need to be customized or programmed specifically to generate random numbers. As such, they are not flexible enough for general consumer-grade devices such as mobile phones or laptops. To make TRNGs

more easily accessible, some recently proposed TRNGs harvest entropy from unpredictable computer behaviour such as race conditions [13], air turbulence in disk drives [14], mouse movement [15, 16], mobile phone cameras [17] and environmental noise recorded via computer microphone [18]. In many of these TRNGs, chaotic maps play a central role in extracting entropy or performing post-processing [4, 5, 7–13, 15–17].

Despite being unpredictable, digitizing physical phenomena to produce random numbers can lead to biased outputs. Therefore, TRNGs require post-processing algorithms to overcome this problem. Most post-processing algorithms such as the XOR corrector [19], Von Neumann corrector [20] and linear codes [21] reduce data rates, leading to reduced performance. More recently, a chaos-based post-processing method based on the logistic map was proposed which does not reduce the data rate [22].

This paper introduces a new chaos-based post-processing algorithm that plays two roles: to eliminate statistical biases, and to provide entropy amplification to boost performance. To fulfil these objectives, the post-processing algorithm is designed based on a hyperchaotic system, specifically a chaotic coupled map lattice (CCML) that consists of multiple unimodal chaotic maps. Due to its sensitivity, changes of even a single bit can be magnified by the hyperchaotic system to produce entirely different random number sequences. The post-processing algorithm can be used on physical entropy sources to produce truly random numbers, or to post-process statistically-biased PRNGs. The feasibility of the proposed post-processing algorithm is depicted by implementing a high speed audio-based TRNG that utilizes the aforementioned algorithm. The TRNG is scrutinized with various statistical test suites in addition to analysis of its backward and forward security. We also analyzed its degree of periodicity by scale index. The thorough analysis ensures that the generator is suitabile for high security applications based on the criteria for physical RNGs proposed by Schindler and Killman [23]. The proposed post-processing method is designed for software-based generators.

The remaining sections in this paper are structured as follows: Section 2 briefly discusses previous work in post-processing algorithms for RNGs. Next, Section 3 provides a brief introduction to chaotic maps that is vital for understanding the rest of the paper. Section 4 describes the proposed post-processing algorithm, followed by Section 5 that describes the proposed audio-based TRNGs. Section 6 evaluates the generator's security and performance. The paper is concluded in Section 7 with some final remarks.

2 Previous work

There exists various post-processing algorithms that serve the purpose of eliminating statistical biases from the outputs of TRNGs. Among the simplest post-processing algorithms that exist are the XOR [19] and Von Neumann [20] correctors. The XOR corrector merely performs the XOR operation on two consecutive blocks of random bits and uses the result as the TRNG's output. However, this reduces the performance of the TRNG by $\frac{1}{2}$ because for every n-bit sequence, the post-processing algorithm will produce a $\frac{n}{2}$ corrected sequence.

The Von Neumann corrector follows a specific rule set to perform post-processing. Consecutive bits in a random sequence are paired and each pair will be either be replaced by a single bit or removed entirely. Although the Von Neumann corrector is effective in eliminating statistical biases, it reduces the performance of the TRNG by $\frac{1}{4}$ in the best case scenario due to its correction algorithm as summarized below:

- $(1, 0) \rightarrow 1$
- $(0, 1) \rightarrow 0$
- $(0, 0)$ or $(1, 1) \rightarrow$ *discard*

Other post-processing methods include the use of resilient functions to eliminate statistical defects [24]. A resilient function ensures that an adversary with knowledge to a subset of inputs to the function will have no advantage in guessing the output. However, in the study conducted by Sunar, the effective throughput of the proposed TRNG is reduced to $\frac{1}{16}$ of its original data rate due to the post-processing. Resilient functions can be obtained from error-correcting codes such as linear codes. Linear codes themselves can be used as post-processing algorithms [21]. It was shown that proper selection of linear code can lead to low bias and a throughput of $\frac{3}{4}$ of the original data rate [25].

Avaroğlu et al. proposed a chaos-based post-processing method, implemented on hardware [22]. The output of a chaotic logistic map is discretized as

$$d(x) = \begin{cases} 1, & x \geqslant 0.5, \\ 0, & x < 0.5, \end{cases} \tag{1}$$

where x is the current state value of the logistic map. The logistic map is iterated to produce a stream of bits that is used to mask the output of the TRNG via XOR operation. The post-processing method does not reduce the data rate of the original TRNG and can successfully eliminate biases. It only introduces additional computational overhead of iterating the

chaotic map. However, this method is less effective when implemented in software because the logistic map needs to be iterated for every single output bit of the TRNG. Implementation of chaotic maps on software requires the use of floating point operations, which can incur significant overhead. Despite this drawback, their work has depicted the feasibility of chaos in post-processing random numbers.

More recently, a new post-processing method proposed by Nikolic and Veinovic was used in their TRNG based on sound cards [18]. The method was known as "Mixing Bits in Steps and XORing of Adjacent Bits" (MiBiS&XOR). After obtaining a sound recording, the bits from the analog-to-digital (ADC) converter are interleaved with one another in a fixed manner. Then, adjacent bits are XORed with each other to obtain the final output. Because the post-processing method is computationally intensive, the authors utilized two independent mixers that work in alternation. MiBiS&XOR was able to pass statistical tests with a throughput of approximately 352.8 Kilobits per second (Kbps).

3 Chaotic maps

Chaotic maps are mathematical functions that depict chaotic behaviour. They have characteristics such as random-like behaviour, sensitivity to initial conditions, diffusion and confusion capabilities. These characteristics are analogous to requirements in cryptography, which is why they have been used to design cryptographic primitives such as hash functions [26, 27], encryption algorithms [28, 29], and RNGs [17, 30]. A popular map that has been used to design many algorithms is the chaotic tent map which is mathematically defined as

$$f_T(x) = \begin{cases} \alpha x, & 0 \leqslant x < 0.5, \\ \alpha(1-x), & 0.5 \leqslant x \leqslant 1, \end{cases} \tag{2}$$

where α is the control parameter. It is a piecewise function that operates on real numbers in the interval of $[0, 1]$. As α is increased from $0 \to 2$, the behaviour of the tent map evolves from periodic to non-periodic as shown in the bifurcation diagram in Fig. 1, where the shaded region indicates chaoticity.

In this work, the tent map will be used as part of a larger chaotic system known as a chaotic coupled map lattice (CCML). It is a spatiotemporal system whereby the state values of multiple simple maps affect one another via nearest neighbour coupling. It can be mathematically defined as

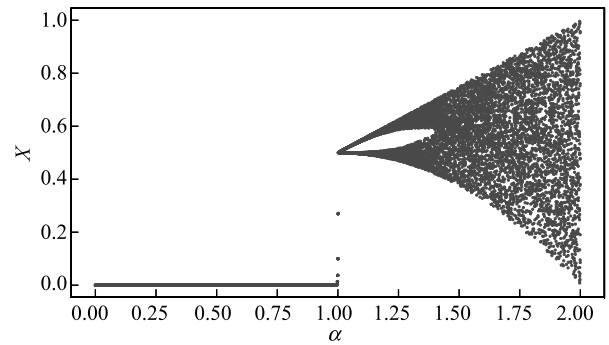

Fig. 1 Tent map bifurcation diagram

$$x_{t+1}^i = (1-\varepsilon)f(x_t^i) + \frac{\varepsilon}{2}(f(x_t^{i+1}) + f(x_t^{i-1})), \tag{3}$$

where ε is a coupling constant within the range of $(0, 1)$, $i = \{0, 1, \ldots, L-1\}$ is the local chaotic map index, $x_t^L \equiv x_t^0$, t is the time index (iterations), L is the number of local maps (lattice size), and f is the local chaotic map function. The CCML is a hyperchaotic system, characterized by having at least two positive Lyapunov exponents and higher complexity than simple chaotic maps. Unlike analogue chaos, the outputs of a chaotic map simulated on a computing device (digital chaos) will eventually fall into a cycle (become periodic) due to round-off errors. In this case, a hyperchaotic map has a longer period as compared to unimodal chaotic maps such as the tent map.

For the proposed algorithm, the tent map with $\alpha = 1.99999$ is used as f and the coupling constant is set to $\varepsilon = 0.05$. $\alpha = 1.99999$ is selected to maximize the phase space of x to include the entire range of $[0, 1]$ as depicted in the tent map bifurcation diagram in Fig. 1. The lattice size, L can be set based on user requirements as described in Section 4.

4 Proposed post-processing algorithm

4.1 Preliminaries

The proposed post-processing algorithm revolves around perturbing the CCML state values with random bits extracted from an entropy source. Then, the system is iterated to diffuse the random bits throughout the system. The final state values of the CCML are combined to produce the final random numbers. As discussed in Section 3, the CCML consists of multiple chaotic tent maps, each with their own state values. The initial states, x_0^i must first be initialized to arbitrarily chosen real numbers within the range of $(0, 1)$. These values can be fixed or dynamically generated by a pseudorandom number generator. A sample set of values that can be used for a CCML with $L = 8$

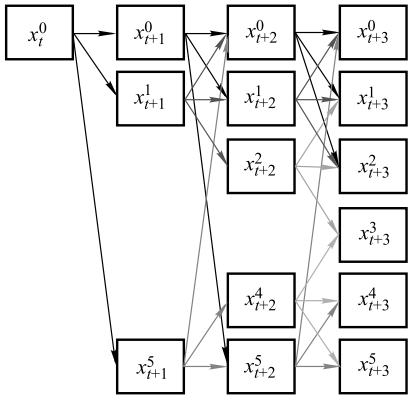

Fig. 2 CCML diffusion for L=6

is $x = \{0.141592, 0.653589, 0.793238, 0.462643, 0.383279, 0.502884, 0.197169, 0.399375\}$. These values are arbitrarily chosen from the digits of π.

Due to the nearest neighbour coupling, the CCML will achieve maximal diffusion in $\lfloor \frac{L}{2} \rfloor$ iterations, where $\lfloor x \rfloor$ is the *floor* function returns the greatest integer that is less than or equal to x. For example, a lattice sizes of $L = 6$ will achieve full diffusion in three iterations, as depicted in Fig. 2.

4.2 Chaotic perturbation

When designing TRNGs based on physical processes, these processes need to be digitized. For example, temperature can be stored as Celsius or Fahrenheit, sound is stored as digital audio samples, and race conditions can be quantified by forcing them to share a single variable [13]. In the ideal case, the digitized data has an entropy level of eight bits per byte. However in practice, it is not uncommon to have an entropy of three to four bits per byte or lower, which implies the existence of statistical bias.

In the proposed post-processing algorithm, bits with higher unpredictability within each byte are first extracted and concatenated to form a b-bit random number, y^i. The location of the random bits within each byte is constant but the exact location can vary depending on the underlying entropy source. The selection of these random bits will depend on how the entropy source is quantized and can be detected by performing entropy analysis. y^i is then used in a perturbation function, p which modifies a chaotic state value x_t^i as

$$p(x_t^i) = (\frac{\omega \times y^i}{2^b - 1} + x_t^i) \times \frac{1}{1+\omega}, \tag{4}$$

where ω is the weight coefficient that ranges between $(0, 1)$. This equation ensures that the perturbed value of x_t^i remains within the range of $[0, 1]$ to prevent the tent map from being iterated out of scope. Each value of x_t^i will be modified by y^i which are each extracted from different bytes. An example of the perturbation process for a CCML with $L = 6$ is depicted in Fig. 3.

4.3 Diffusion and final output

After the chaotic perturbation has been performed, the CCML is iterated $\lfloor \frac{L}{2} \rfloor$ times to diffuse the bits throughout the system. The state values will be affected by $L \times b$ bits each after completing all the iterations. The final output bits is calculated from the L states as

$$z^i = z^i \boxplus swap(z^{i+\frac{L}{2}}), \quad 0 \leqslant i < \frac{L}{2}, \tag{5}$$

where z^i is the binary representation of the real number $x^i_{\lfloor \frac{L}{2} \rfloor}$, $swap(x)$ is a function that swaps the bits from the first half of x with the second half. $\boxplus$ is a 32-bit or 64-bit modular addition depending on whether single-precision or double-precision floating point format is used to represent real numbers. The final random number sequence is a concatenation of the z values, $o = z^0 \| z^1 \| \cdots \| z^i$ for $0 \leqslant i < \frac{L}{2}$. Using Eq. (5) to formulate the final random output maximizes the number of random bits that can be generated from each chaotic state. The *swap* function is important because the most significant bits (MSB) of the floating point representation has lower randomness compared to the least significant bits (LSB).

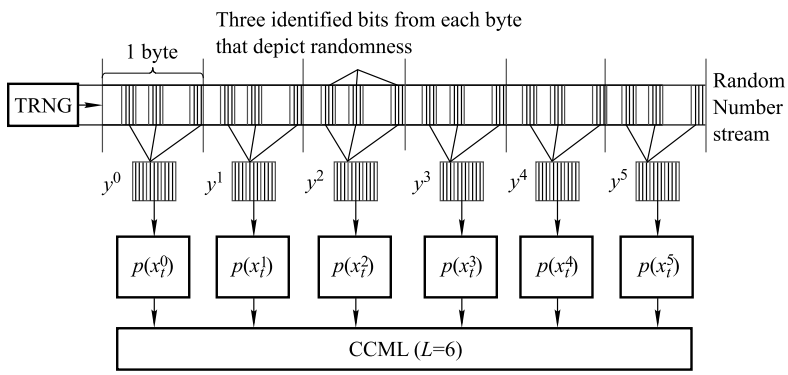

Fig. 3 Perturbation process

Selection of the lattice size is a trade-off between security and performance. Increasing the lattice size improves the mixing capability of the CCML but reduces the performance. To produce an R-bit random number sequence, the total number of individual chaotic map iterations required can be calculated as

$$T = \frac{2R}{s} \times (\lfloor \frac{L}{2} \rfloor + 1), \tag{6}$$

where s is the number of bits required to represent the floating point number. For example, to generate a 32768-bit random number sequence using the post-processing algorithm with $L = 8$ and $s = 64$, the number of iterations required is 5120 iterations of the tent map. For $L = 5$ and $s = 64$, the number of tent map iterations is 3072 which is a 40% increase in performance. Thus, a TRNG that utilizes the proposed post-processing algorithm can be easily modified to suit various applications by changing the lattice size.

5 Proposed audio-based true random number generator

To depict the feasibility of the proposed post-processing algorithm, it is used to design an audio-based TRNG. A computer microphone is used to record and digitize environmental sound using the Waveform Audio Interface (WAV) API. A sampling rate of 44.1 kHz is used in this work, where 44100 samples are recorded per second and stored as 8-bit variables. To ensure that the recorded samples are truly unpredictable, the first 10000 samples are discarded to eliminate any transient effect that may occur at the beginning of the recording. Figure 4 shows the transient effect that occurs when audio is recorded using a computer microphone. A short period of non-random behaviour prior to the 5000th sample can be observed. Figure 5 depicts another audio recording where the first 10000 samples has been skipped. As expected, the transient effect no longer exists.

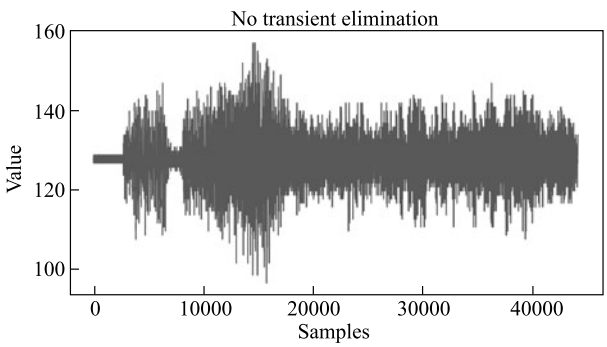

Fig. 4 Audio recording without transient elimination

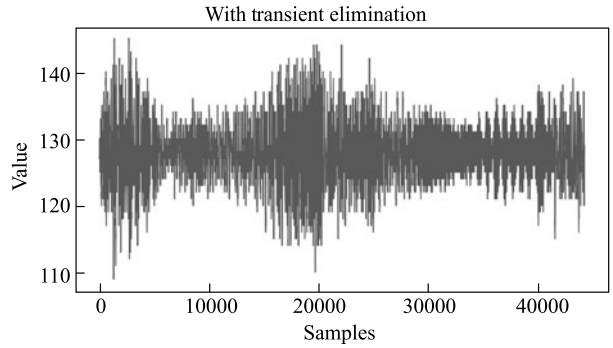

Fig. 5 Audio recording with transient elimination

In quiet environments, not all bits of each 8-bit sample display erratic behaviour. Therefore, bits with sufficient entropy have to be identified for use in the proposed TRNG. Entropy assessment is used to achieve this goal. Experiments were performed in three different environments: silent, normal and noisy. The three least significant bits (LSB) of samples recorded in all three environments were identified to have random-like patterns. To form a 1-Megabit sequence (one for each environment), three LSBs from approximately 349526 samples are concatenated. The entropy for the 1-Megabit sequence is then calculated. The entropy for the quiet, normal and noisy environments are 3.871733, 6.703212 and 7.165172 bits per byte respectively. All three do not meet the ideal entropy value of 8.0 but depict non-deterministic behaviour nonetheless.

To amplify the entropy obtained from the audio recordings, the previously described post-processing algorithm is used. Two variants of the TRNG are proposed in this work using different lattice sizes for the CCML ($L = 6$ and $L = 8$). The TRNGs are denoted as $TRNG_6$ and $TRNG_8$ respectively. The states of the CCML are represented as 64-bit IEEE double-precision floating point (FP) format ($s = 64$), thus the CCML state sizes are 384 and 512 bits for $TRNG_6$ and $TRNG_8$ respectively. The initial states are initialized with the arbitrarily chosen FP values as discussed in Section 1. The parameters for the chaotic perturbation are $\omega = \frac{1}{2}$ and $b = 3$. The number of audio samples, n that needs to be recorded is dependent on the required length of the final true random number sequence (TRNS), N and also the number of output bits produced at one time, calculated as $B = \frac{L \times s}{2}$ bits. $B = 192$ and $B = 256$ bits for $TRNG_6$ and $TRNG_8$ respectively. The pseudocode of the TRNG is as shown in Algorithm 1. The overall flow of the generator is summarized below:

1) Identify the required length of the TRNS, N and calculate the number of required samples n.
2) Record n samples using the microphone.

3) Extract and store three LSBs from each sample.
4) Initialize the CCML states.
5) Perturb the CCML states using Eq. (4).
6) Iterate the CCML $\lfloor \frac{L}{2} \rfloor$ times.
7) Store the binary representation of the CCML states in z^i and calculate Eq. (5).
8) Concatenate the z values to form the final B-bit random number, stored in an array O.
9) Repeat steps 5 to 8 until the TRNS length is N bits.

Algorithm 1 TRNG algorithm

Input: Size of required TRNS (bits): N, CCML state variable: **double** x_t^i, CCML size: L, coupling constant: ε, weight coefficient: ω, variable size in bits: s.
Output: Array of B-bit random numbers: O

$\gamma = flr(\frac{L}{2})$ //Number of iterations
$n = \frac{N}{\frac{L}{2} \times s} \times L = \frac{2N}{s}$ //Number of samples
$A = rec(n)$ //Record audio clip with n-samples and store in array A
for $i = 0$ **to** n **do**
  $y_i = extract(A[i])$ //Extract 3 bits from each sample
end for
$x_0^0 = 0.141592$, $x_0^1 = 0.653589$, $x_0^2 = 0.793238$, $x_0^3 = 0.462643$
$x_0^4 = 0.383279$, $x_0^5 = 0.502884$
if $L = 8$ **then**
  $x_0^6 = 0.197169$, $x_0^7 = 0.399375$
end if
$c = 0$ //Set a counter to 0
while $sizeof(O) \leqslant N$ **do**
  for $i = 0$ **to** $(L-1)$ **do**
    $x_t^i = (\frac{\omega \times y_c}{2^b - 1} + x_t^i) \times \frac{1}{1+\omega}$
    $c = c + 1$
  end for
  for $t = 0$ **to** $(\gamma - 1)$ **do**
    for $i = 0$ **to** $(L-1)$ **do**
      $x_{t+1}^i = (1-\varepsilon) f_T(x_t^i) + \frac{\varepsilon}{2}(f_T(x_t^{i+1 \mod L}) + f_T(x_t^{i-1 \mod L}))$
    end for
  end for
  for $i = 0$ **to** $(L-1)$ **do**
    $z^i = uint64(x_{\gamma-1}^i)$ //Cast $x_{\gamma-1}^i$ as an unsigned 64-bit integer z^i
    $x_0^i = x_{\gamma-1}^i$ //Set initial value for next round of iterations
  end for
  for $i = 0$ **to** $(\frac{L}{2} - 1)$ **do**
    $z^i = z^i \boxplus swap(z^{i+\frac{L}{2}})$
  end for
  $O \leftarrow z^0 \| \ldots \| z^{\frac{L}{2}}$ //Concatenate and store the B-bit random number
end while

6 Evaluation results

To ensure that the proposed TRNGs have proper security analysis, they are evaluated based on the criteria for physical RNGs proposed by Schindler and Killmann [23]. They introduced two types of functionality classes for RNGs depending on their level of security. The first functionality class is the P1 class which is only required to be statistically inconspicuous. Testing for statistical defects is performed using statistical test suites which we have performed in Section 6.1. The P2 class of generators require forward and backward security in addition to fulfilling the requirement of P1 generators. We analyze forward and backward security in Section 6.2. Additional analysis on sensitivity, periodicity, complexity, and performance are provided in Sections 6.3 to 6.6, respectively

6.1 Statistical test suites

The two variants of the proposed TRNG are tested using three statistical test suites, NIST SP 800-22 [31], DIEHARD [32], and ENT [33]. These test suites can detect statistical flaws but cannot differentiate between truly random and pseudo-random numbers. If the TRNGs pass all three test suites successfully, they are considered to have fulfilled the criteria for the P1 class of generators. The sample sizes and passing requirements have been obtained from the documentation of the respective test suites.

The NIST test suit comprises of 15 statistical tests. The recommended number of samples for each of these tests is 1000, where each sample size is 1-Megabit. Each test generates a P-value which indicates if the sample passes or fails the test. A RNG is considered to have successfully passed the NIST test suite if the passing ratio for each of the tests is higher than $p' = 0.980561$ with the exception of the random excursion tests which require $p' = 0.978060$. Also, the overall P-value calculated from the 1000 P-values of each test must be larger than 0.0001. Both $TRNG_6$ and $TRNG_8$ successfully passed the NIST test suite as shown in Table 1.

The second test is the DIEHARD test suite which has 18 individual statistical tests. Each test produces P-values that range from 0 to 1. A random number sequence is considered to have failed a test if the P-value falls below 0.01 or is higher than 0.99. The recommended input size for this test suite is 87.5 Megabits. Some tests produce multiple P-values, so only the worst case results are listed in Table 2. Both generators successfully passed the DIEHARD test suite.

The ENT test suite has five statistical test for randomness. Each individual test has different measures and do not produce P-values unlike the previous test suites. Experimental results are compared against the ideal results of each test to determine if a random number sequence has observable statistical defects. Both generators have near-ideal characteristics as shown in Table 3.

Table 1 NIST test results

Test name	TRNG$_6$ P-value	p'	TRNG$_8$ [34] P-value	p'
Frequency	0.165340	0.989	0.796268	0.991
Block frequency	0.328297	0.990	0.437274	0.990
Cumulative sums	0.045971	0.990	0.395940	0.993
Runs	0.163513	0.995	0.678686	0.990
Longest run	0.240501	0.991	0.064822	0.987
Rank	0.668321	0.985	0.467322	0.992
FFT	0.992084	0.990	0.643366	0.987
Non-overlapping templates	0.643366	0.983	0.987896	0.982
Overlapping templates	0.651693	0.993	0.117432	0.984
Universal	0.502247	0.988	0.919131	0.987
Approximate entropy	0.390721	0.995	0.461612	0.994
Random excursions	0.245004	0.979	0.967842	0.980
Random excursions variant	0.516370	0.982	0.002472	0.988
Serial	0.183547	0.987	0.322135	0.984
Linear complexity	0.213309	0.995	0.154629	0.993

*To pass, P-value$\geqslant$ 0.0001, $p' \geqslant$ 0.980561/0.978060 (random excursion)

Table 2 DIEHARD test results

Test name	TRNG$_6$ P-value	TRNG$_8$ [34] P-value
Birthday spacings	0.395837	0.064600
Overlapping 5-permutation	0.578730	0.125674
Binary rank for 31×31 matrices	0.543509	0.877028
Binary rank for 32×32 matrices	0.700531	0.656309
Binary rank for 6×8 matrices	0.248942	0.829382
Bitstream	0.02605	0.01281
Overlapping-pairs-sparse occupancy	0.0574	0.0680
Overlapping-quadruples-sparse occupancy	0.0132	0.1047
DNA spacings	0.0419	0.0448
Count-the-1s on a stream of bytes	0.018326	0.936210
Count-the-1s for specific bytes	0.989610	0.031879
Parking lot	0.198285	0.100920
Minimum distance	0.635419	0.902581
3D-spheres	0.449782	0.418768
Squeeze	0.906991	0.922773
Overlapping sums	0.977225	0.477592
Runs	0.334719	0.880336
Craps	0.254498	0.906958

*To pass, 0.01 $\leqslant P$-value$\leqslant$ 0.99

Table 3 ENT test results

Test name	TRNG$_6$ P-value	TRNG$_8$ [34] P-value	Ideal value
Entropy	7.999984	7.999998	8
Chi-square	40.71%	44.19%	50%
Arithmetic mean	127.4892	127.5184	127.5
Monte carlo value for π	3.140863462	3.140933713	3.141592653
Serial correlation coefficient	–0.000433	0.000376	0

6.2 Analysis of forward and backward security

Both $TRNG_6$ and $TRNG_8$ have fulfilled the requirement for P1 generators as they have sufficient statistical randomness. To depict characteristics that qualify these generators for the P2 class, the generators must have forward and backward security, whereby past or future outputs cannot be predicted by an adversary with the power to corrupt the internal state of the generator [35]. In other words, the generator is non-deterministic. Forward and backward security can be analyzed by first proving that the generator is independent and memoryless (Markovian), followed by showing an increase of entropy per internal random number that is sufficiently large [23].

The Markovian property is depicted in the following experiment as proposed in [13]: a generate-reset procedure is used to generate a 1-Megabyte sequence. This generate-reset procedure requires a generator to be reset to its initial state after generating a random number block. The size of this random number block is set to the value of B. To produce a 1-Megabyte sequence, the generate-reset procedure is repeated a total of $\frac{1024\times1024\times8}{B}$ times. The Average Shannon Entropy (ASE) is calculated for this random number sequence based on

$$H(X) = \lim_{k\to\infty} -\frac{1}{k} \sum_{\sigma\in\{0,1\}^k} P(\sigma_k)\log_2 P(\sigma_k), \tag{7}$$

where $P(\sigma_k)$ is the probability mass function of σ_k and the summation includes all binary k-tuples of the form $\sigma_k = b_0,\ldots,b_{k-1}, b \in 0,1$ are collected from the entire random number sequence. As each number is represented by one byte, $k = 8$ and the maximum value for ASE is 1. If this experiment is performed on a PRNG, it will produce the output repeatedly after every reset which leads to a low ASE value. The ASE of the 1-Megabyte file produced by $TRNG_6$ and $TRNG_8$ are 0.9703 and 0.9836 respectively. These results indicate that both TRNGs are non-deterministic and have Markovian characteristics.

Next, both generators need to provide evidence of an increase of entropy per internal random number. For a fixed block size, the Coron test [36] can be used for this purpose. The test is able to calculate the exact information of entropy per bit, but should only be applied on generators that have already shown to be at least Markovian. If the Coron test is applied on its own, the results are out of context and does not signify that the generator has forward or backward security. Because both proposed TRNGs already depict Markovian characteristics, the Coron test is applied with the suggested parameters of $l = 8$, $q = 2570$ and $k = 256000$. To

fulfil the security criteria for P2 generators, the result of the Coron test must be $f > 7.976$. The $TRNG_6$ and $TRNG_8$ generators achieve results of 7.9979 and 7.9977 respectively, indicating that they are both P2 class generators which can be used for high security applications.

6.3 Sensitivity

The proposed TRNGs are highly sensitive to changes in the audio sample bits, whereby the change in even one bit will generate an entirely different output sequence. This is depicted in the following experiment: two 1-Megabit sequences are generated by the TRNG based on a simulated audio signal of all zero bits (0,0,0,...,0) and another that starts with one followed by all zero bits (1,0,0,...,0). The XOR difference of both sequences is taken as the error signal. The entropy for this error signal is calculated as 7.99966 and 7.99889 bits per byte for $TRNG_6$ and $TRNG_8$ respectively, indicating that even an input difference of one bit leads to near maximal difference.

6.4 Degree of non-periodicity

To measure the degree of non-periodicity of the proposed TRNGs, we leverage upon the methodology proposed by Benítez et al. [37] known as the scale index. This technique is based on the continuous wavelet transform (CWT) and wavelet multiresolution analysis, whereby the scale index parameter can be used as a measure of a signal's non-periodicity. The CWT of a signal f at time u and scale s is defined as

$$Wf(u,s) := \langle f, \psi_{u,s} \rangle = \int_{\infty}^{\infty} f(t)\psi^*_{u,s}\mathrm{d}t, \tag{8}$$

which provides the details (frequency component) of f that corresponds to s and time location, t. The scalogram of f, ς is defined as

$$\varsigma(s) := \|Wf(u,s)\| = \left(\int_{\infty}^{\infty} |Wf(u,s)|^2 \mathrm{d}u\right)^{\frac{1}{2}}, \tag{9}$$

where $\varsigma(s) \geqslant 0$ is the energy of a CWT of f at scale s. If f has a scalogram of $\varsigma(s) > 0$, we denote it has having details at scale s. Next, the inner scalogram of f is defined as

$$\varsigma^{inner}(s) := \|Wf(s,u)\|_{J(s)} = \left(\int_{c(s)}^{d(s)} |Wf(s,u)|^2 \mathrm{d}u\right)^{\frac{1}{2}}, \tag{10}$$

where $J(s) = [c(s), d(s)] \subseteq I$ is the maximal subinterval in I for which the support of $\psi_{u,s}$ is included in I for all $u \in J(s)$. The values of the inner scalogram for different s cannot be compared because the length of $J(s)$ is dependent on s. Therefore, the inner scalogram should be normalized as

$$\bar{\varsigma}^{inner}(s) = \frac{\varsigma^{inner}(s)}{(d(s) - c(s))^{\frac{1}{2}}}. \tag{11}$$

The normalized inner scalogram can be used to analyze non-periodicity, whereby a signal with details at every scale is non-periodic. Thus, the selection of the scale interval, $[s_0, s_1]$ is vital for scalogram analysis. The selection of s_1 is especially crucial because the non-periodic behavior of a signal is based on its properties at larger scales. For our analysis, we select integer scales of $s_0 = 1$ and $s_1 = 64$, similar to the experiments conducted in [37]. The scale index of f can then be calculated as

$$i_{scale} := \frac{\varsigma(s_{min})}{\varsigma(s_{max})}, \tag{12}$$

where s_{max} is the smallest scale where $\varsigma(s) \leqslant \varsigma(s_{max})$ for all $s \in [s_0, s_1]$, and s_{min} is the smallest scale where $\varsigma(s_{min}) \leqslant \varsigma(s)$ for all $s \in [s_{max}, s_1]$. Thus, i_{scale} will fall within the range of [0,1]. When $i_{scale} \to 0$, f is periodic whereas when $i_{scale} \to 1$, f is highly non-periodic.

For $TRNG_6$ and $TRNG_8$, we obtain $i_{scale} = 0.9154650$ and $i_{scale} = 0.9083690$ respectively. Both scale indexes are close to the value of 1, implying a high degree of non-periodicity. Visual depiction of the scale indexes for the entire scale interval of $[s_0, s_1]$ are shown in Figs. 6 and 7 for $TRNG_6$ and $TRNG_8$ respectively.

6.5 Additional complexity measures

To analyze the complexity of the random sequences generated by the proposed TRNG, we observe two definitions of complexity: (1) The number of bits that is required by a machine to reproduce the original sequence [38] and (2) linear complexity that refers to the shortest linear feedback shift register (LFSR) able to reproduce the original sequence [39]. Huffman compression is first employed to obtain a coarse-grained measure of the sequence complexity based on

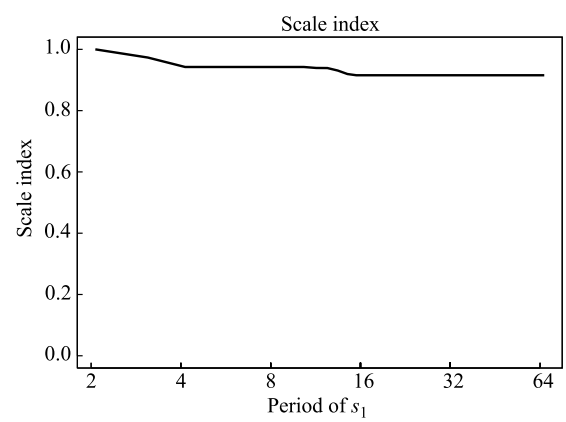

Fig. 6 Scale indexes for $TRNG_6$

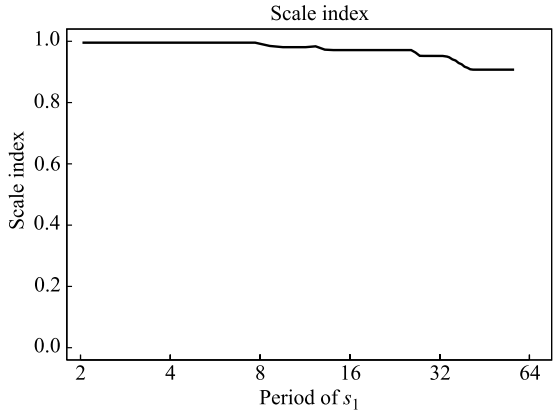

Fig. 7 Scale indexes for $TRNG_8$

definition (1). 128 kilobytes of random numbers generated by both $TRNG_6$ and $TRNG_8$ were compressed using Huffman compression, to produce outputs of 129 Kilobytes. The resulting file sizes are larger due to additional bytes required to store coding information. This preliminary result indicates that the random number sequences have maximal theoretical complexity because they cannot be compressed.

Next, we use the Berlekamp-Massey algorithm to analyze the linear complexity profile of the random number sequences generated from the proposed TRNGs [40, 41]. A truly random sequence will have a linear complexity of $\frac{N}{2} \pm 1$, where N is its length in bits. In other words, a LFSR with a length of of $\frac{N}{2} \pm 1$ is required to reproduce an N-bit sequence generated by an ideal TRNG. 100000-bit sequences were generated by both $TRNG_6$ and $TRNG_8$ and fed as inputs to the Berlekamp-Massey algorithm. Results indicate that minimum LFSR lengths of 50001 are required to reproduce sequences from both TRNGs, depicting ideal linear complexity profiles.

6.6 Performance

To analyze performance, we measure the amount of time required to generate a 1-Megabyte random number file that consists of 1048576 8-bit numbers. We observe that the performance of the proposed TRNGs are severly dampened the time required to record the audio samples. Taking into account the recording time, $TRNG_6$ and $TRNG_8$ achieve throughputs of 1.311 and 1.293 Megabits per second which are at least 61 and 3.75 times faster than the sound-based generators in [42] and [18] respectively. Without the use of the proposed post-processing algorithm, the proposed generators will be approximately 11 times slower in addition to having statistical bias. In practice, it is possible to record and store the samples in advance as an entropy pool which eliminates the recording overhead. Samples that are used to generate outputs need to be discarded and new samples can be recorded to replenish the entropy pool. This increases both TRNG speeds to 500.015 and 186.502 Megabits per second. As long as the recorded samples are non-deterministic in nature (which is the case unless the recording device or entropy pool is tampered with), the proposed generator will still generate truly random outputs.

7 Conclusion

This paper introduced a new chaos-based post-processing algorithm that not only eliminates statistical bias but provides entropy amplification to achieve higher performance. The algorithm utilizes the inherent characteristics of hyperchaos such as sensitivity to input changes, random-like behaviour, diffusion and confusion capabilities to achieve both these goals. Random bits generated by an entropy source is used to perturb the states of a hyperchaotic system (CCML) which is iterated to ensure maximal diffusion. The algorithm is flexible enough to post-process any type of entropy source as long as there exists some semblance of uncertainty. Two variants of audio-based TRNGs were implemented with the post-processing algorithm to depict its effectiveness. Results indicate that the TRNGs have no statistical defects, have forward and backward security in addition to outperforming other audio-based generators that in recent literature. This reflects the practical application of the proposed post-processing algorithm when it used to design TRNGs.

Acknowledgements This work was supported in part by the Ministry of Education Malaysia under the Fundamental Research Grant Scheme (FRGS/1/2019/ICT05/USM/02/1), Universiti Sains Malaysia (304/PKOMP/6315190) and the National Natural Science Foundation of China (Grant No. 61702212).

References

1. Cret O, Gyorfi T, Suciu A. Implementing true random number generators based on high fanout nets. Romanian Journal of Information Science and Technology, 2012, 15(3): 277–298
2. Jun B, Kocher P. The intel random number generator. Cryptography Research Inc. White Paper, 1999, 27: 1–8
3. Cicek I, Pusane A E, Dundar G. An integrated dual entropy core true random number generator. IEEE Transactions on Circuits and Systems II: Express Briefs, 2017, 64(3): 329–333
4. Karakaya B, Çelik V, Gulten A. Chaotic cellular neural network-based true random number generator. International Journal of Circuit Theory and Applications, 2017, 45(11): 1885–1897
5. Bonny T, Debsi R A, Majzoub S, Elwakil A S. Hardware optimized FPGA implementations of high-speed true random bit generators based

on switching-type chaotic oscillators. Circuits, Systems, and Signal Processing, 2018, 38(3): 1342–1359

6. Mei F, Zhang L, Gu C, Cao Y, Wang C, Liu W. A highly flexible lightweight and high speed true random number generator on FPGA. In: Proceedings of IEEE Computer Society Annual Symposium on VLSI (ISVLSI). 2018
7. Nguyen T T N, Kaddoum G, Gagnon F. Implementation of a chaotic true random number generator based on fuzzy modeling. In: Proceedings of the 16th IEEE International New Circuits and Systems Conference. 2018
8. Kumar D, Nabi K, Misra P K, Goswami M. Modified tent map based design for true random number generator. In: Proceedings of IEEE International Symposium on Smart Electronic Systems. 2018
9. Alcin M, Koyuncu I, Tuna M, Varan M, Pehlivan I. A novel high speed artificial neural network-based chaotic true random number generator on field programmable gate array. International Journal of Circuit Theory and Applications, 2018, 47(3): 365–378
10. Hsueh J C, Chen V H C. An ultra-low voltage chaos-based true random number generator for IoT applications. Microelectronics Journal, 2019, 87: 55–64
11. Gupta R, Pandey A, Baghel R K. FPGA implementation of chaos-based high-speed true random number generator. International Journal of Numerical Modelling: Electronic Networks, Devices and Fields, 2019, 32(5): e2604
12. Karakaya B, Gulten A, Frasca M. A true random bit generator based on a memristive chaotic circuit: analysis, design and FPGA implementation. Chaos, Solitons & Fractals, 2019, 119: 143–149
13. Teh J S, Samsudin A, Al-Mazrooie M, Akhavan A. GPUs and chaos: a new true random number generator. Nonlinear Dynamics, 2015, 82(4): 1913–1922
14. Davis D, Ihaka R, Fenstermacher P. Cryptographic randomness from air turbulence in disk drives. In: Proceedings of Annual International Cryptology Conference. 1994, 114–120
15. Hu Y, Liao X, wo Wong K, Zhou Q. A true random number generator based on mouse movement and chaotic cryptography. Chaos, Solitons & Fractals, 2009, 40(5): 2286–2293
16. Xingyuan W, Xue Q, Lin T. A novel true random number generator based on mouse movement and a one-dimensional chaotic map. Mathematical Problems in Engineering, 2012
17. Yeoh W Z, Teh J S, Chern H R. A parallelizable chaos-based true random number generator based on mobile device cameras for the android platform. Multimedia Tools and Applications, 2019, 78(12): 15929–15949
18. Nikolic S, Veinovic M. Advancement of true random number generators based on sound cards through utilization of a new post-processing method. Wireless Personal Communications, 2016, 91(2): 603–622
19. Davies R B. Exclusive OR (XOR) and hardware random number generators. see Wikipedia, 2002
20. Von Neumann J. Various techniques used in connection with random digits. National Bureau of Standards Applied Mathematical Series, 1951, 12(36–38): 5
21. Lacharme P. Post-processing functions for a biased physical random number generator. In: Proceedings of International Workshop on Fast Software Encryption. 2008, 334–342
22. Avaroğlu E, Tuncer T, Őzer A, Ergen B, Tűrk M. A novel chaos-based post-processing for TRNG. Nonlinear Dynamics, 2015, 81(1–2): 189–199
23. Schindler W, Killmann W. Evaluation criteria for true (physical) random number generators used in cryptographic applications. In: Proceedings of International Workshop on Cryptographic Hardware and Embedded Systems. 2002, 431–449
24. Sunar B, Martin W J, Stinson D R. A provably secure true random number generator with built-in tolerance to active attacks. IEEE Transactions on Computers, 2007, 56(1): 109–119
25. Kwok S H, Ee Y L, Chew G, Zheng K, Khoo K, Tan C H. A comparison of post-processing techniques for biased random number generators. In: Proceedings of IFIP International Workshop on Information Security Theory and Practices. 2011, 175–190
26. Ahmad M, Khurana S, Singh S, AlSharari H D. A simple secure hash function scheme using multiple chaotic maps. 3D Research, 2017, 8(2): 13
27. Li Y, Ge G. Cryptographic and parallel hash function based on cross coupled map lattices suitable for multimedia communication security. Multimedia Tools and Applications, 2019, 78(13): 17973–17994
28. ur Rehman A, Liao X. A novel robust dual diffusion/confusion encryption technique for color image based on chaos, DNA and SHA-2. Multimedia Tools and Applications, 2018, 78(2): 2105–2133
29. Xiong Z, Wu Y, Ye C, Zhang X, Xu F. Color image chaos encryption algorithm combining CRC and nine palace map. Multimedia Tools and Applications, 2019, 78(22): 31035–31055
30. Garcia-Bosque M, Perez-Resa A, Sanchez-Azqueta C, Aldea C, Celma S. Chaos-based bitwise dynamical pseudorandom number generator on FPGA. IEEE Transactions on Instrumentation and Measurement, 2019, 68(1): 291–293
31. Rukhin A, Soto J, Nechvatal J. A statistical test suite for random and pseudorandom number generators for cryptographic applications. National Institute of Standards, NIST Special Publication 800-22, 2010
32. Marsaglia G. DIEHARD: a battery of tests of Randomness. 1996
33. Walker J. ENT Program. 2008
34. Teh J S, Teng W, Samsudin A. A true random number generator based on hyperchaos and digital sound. In: Proceedings of the 3rd International Conference on Computer and Information Sciences. 2016, 264–269
35. Dodis Y, Pointcheval D, Ruhault S, Vergniaud D, Wichs D. Security analysis of pseudo-random number generators with input: /dev/random is not robust. In: Proceedings of the 2013 ACM SIGSAC Conference on Computer & Communications Security. 2013, 647–658
36. Coron J S. On the security of random sources. In: Proceedings of International Workshop on Public Key Cryptography. 1999, 29–42
37. Benítez R, Bolós V, Ramírez M. A wavelet-based tool for studying non-periodicity. Computers & Mathematics with Applications, 2010, 60(3): 634–641
38. Ritter T. The efficient generation of cryptographic confusion sequences. Cryptologia, 1991, 15(2): 81–139
39. Golomb S W. Shift register sequences. World Scientific. 2014
40. Massey J. Shift-register synthesis and BCH decoding. IEEE Transactions on Information Theory, 1969, 15(1): 122–127
41. Menezes A J, van Oorschot P C, Vanstone S A. Handbook of Applied Cryptography. CRC Press, 2018

42. Bardis N G, Markovskyi A P, Doukas N, Karadimas N V. True random number generation based on environmental noise measurements for military applications. In: Proceedings of the 8th WSEAS International Conference on Signal Processing, Robotics and Automation. 2009, 68–73

Je Sen Teh received the BEng degree (Hons.) majoring in Electronics from Multimedia University, Malaysia in 2011, then received his MSc and PhD in Computer Science from Universiti Sains Malaysia in 2013 and 2017, respectively. He is currently working as a Senior Lecturer in Universiti Sains Malaysia under the School of Computer Sciences. His research interests include symmetric cryptography, cryptanalysis, random number generation and chaos theory.

Weijian Teng graduated from Multimedia University, Malaysia with a BEng (hons) in Electronics Engineering in 2011. He subsequently pursued his MSc in Acoustics and Music Technology from the University of Edinburgh, UK in 2012. Since then, he has been working as a lecturer in private institutions of higher learning and is now attached to INTI International College Penang, Malaysia. His areas of interest include areas of applied mathematics and sciences like audio and digital signal processing as well as heuristics and algorithms.

Azman Samsudin is a Professor at the School of Computer Science, Universiti Sains Malaysia (USM), Malaysia. He earned his BSc in Computer Science from University of Rochester, USA in 1989. Later, he received his MSc and PhD in Computer Science, in 1993 and 1998, respectively, both from the University of Denver, USA. His research interests include Cryptography, Switching Networks and Parallel Computing. He has published more than 100 articles over a series of books, professional journals and conference proceedings.

Jiageng Chen received the BS degree from the School of Compuer Science and Technology, Huazhong University of Science and Technology (HUST), China in 2004 and recieved his MS and PhD of computer science from the School of Information Science, Japan Advanced Institute of Science and Technology (JAIST), Japan in 2007 and 2012, respectively. He was working as an Assistant Professor in School of Information Science, Japan Advanced Institute of Science and Technology, Japan from 2012 to 2015. And currently, he is an Associate Professor at the Computer School of Central China Normal University, China. His research areas include cryptography, especially in the areas of algorithms, cryptanalysis, secure designs and so on.